# Hybrid Parallel Graph Processing

## Using MPI and OpenMP

*Accelerating Breadth-First Search on Large-Scale Road Networks*


**Team Members**

| | |
|---|---|
| Gagan | U23AI060 |
| Harshit | U23AI075 |
| Diwyanshu | U23AI089 |
| Tarachand | U23AI099 |


**High Performance Computing**

MPI

OpenMP

BFS

Graph Processing

# Problem Statement

*Why is BFS on large graphs hard?*

> *"Graph traversal on 1.96M vertices with 5.5M edges is computationally intensive. How can we exploit both distributed and shared memory to maximize throughput?"*

### Serial BFS Too Slow

Single-threaded BFS takes ~95 seconds on roadNet-CA — unacceptable for real-time applications.

### Memory Bottleneck

No single processor can efficiently cache the full working set of a 1.96M vertex graph.

### Scale Limitation

Scaling requires BOTH distributed memory (nodes) AND shared memory (cores) simultaneously.

## Our Solution: Two-Level Hybrid Parallel BFS

**LEVEL 1 — MPI**

Distributes vertex ownership across independent processes (nodes/sockets)

**LEVEL 2 — OpenMP**

Parallelizes frontier expansion within each process across CPU cores

**DATASET**

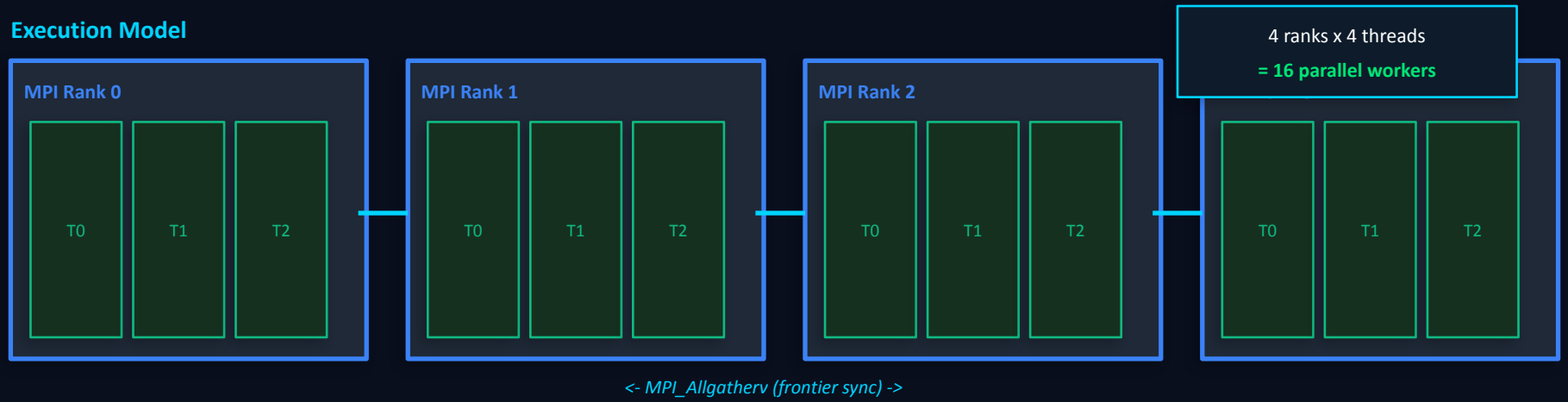
roadNet-CA (Stanford SNAP) — 1.96M vertices, 5.53M arcs

# Why Hybrid Parallelism?

*MPI + OpenMP — better together*

| MPI Only | OpenMP Only | Hybrid MPI+OpenMP |
|---|---|---|
| Scales across nodes<br>Distributed memory | Multi-core on shared mem<br>Low communication cost | P ranks x T threads workers<br>Maps to NUMA architecture<br>Unlimited nodes + all cores |
| Wastes extra cores per socket<br>1 rank = 1 core used | Cannot exceed single node<br>Memory hard ceiling | Best of both worlds! |

## Execution Model



# Dataset — California Road Network

*Stanford SNAP · roadNet-CA*

| 1.96M | 5.53M | 2.82 | ~73 MB |
|---|---|---|---|
| Vertices | Directed Arcs | Avg Degree | File Size |

### File Format (SNAP edge list)

```
# Nodes: 1965206  Edges: 2766607
# FromNodeId    ToNodeId
0    1
1    2
1    3
2    4
...
```

### Why roadNet-CA?

- Real-world sparse graph — stresses parallel BFS fairly
- 1.96M vertices — large enough to benchmark communication
- Low avg degree (2.82) — predictable frontier expansion
- Publicly available & reproducible results

### Alternative Datasets Supported

**roadNet-PA**

Vertices: 1.09M · Edges: 1.54M

**roadNet-TX**

Vertices: 1.38M · Edges: 1.92M

**Custom (test_graph.txt)**

Vertices: Variable · Edges: Variable

*Source: Stanford Network Analysis Project (SNAP) · snap.stanford.edu/data/roadNet-CA.html*

# Project Architecture & Folder Structure

*Module responsibilities and data flow*

```
HPC_Project/project/
+-- CMakeLists.txt
+-- README.md
+-- src/
|   +-- main.cpp
|   +-- graph_io.hpp/.cpp
|   +-- partition.hpp/.cpp
|   +-- parallel_bfs.hpp/.cpp
|   +-- serial_bfs.hpp/.cpp
|   +-- benchmark.hpp/.cpp
+-- data/ (roadNet-CA.txt)
+-- dashboard/ (app.py)
+-- build/ -> hybrid_bfs
+-- results/ -> benchmark.csv
```

| Module | Language | Responsibility |
|---|---|---|
| `main.cpp` | C++/MPI | Init, CLI args, broadcast, lifecycle |
| `graph_io` | C++ | Parse SNAP -> Build CSR |
| `partition` | C++ | Assign vertex ranges to ranks |
| `parallel_bfs` | MPI+OMP | Core hybrid BFS algorithm |
| `serial_bfs` | C++ | Baseline for speedup measurement |
| `benchmark` | C++/MPI | Timing, TEPS, CSV output |
| `app.py` | Python | Interactive Streamlit dashboard |

# CSR — Compressed Sparse Row Graph Representation

*Memory-efficient graph storage*

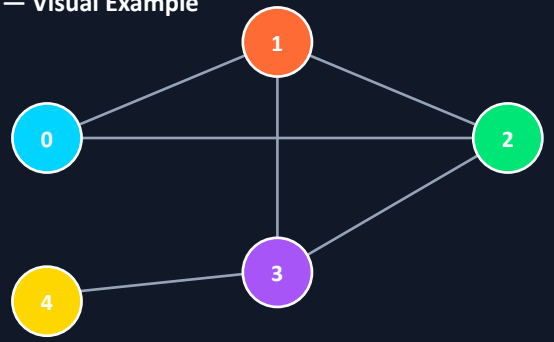
### Why NOT Adjacency Matrix?

1.96M x 1.96M = ~14.9 trillion cells · Most entries are zero · Completely infeasible for sparse graphs

### CSR Data Structure

```
struct CSRGraph {
  int num_vertices;  // 1,965,206
  int num_edges;     // 5,533,214
  vector<int> offsets;   // V+1
```

### CSR Layout — Visual Example




**offsets[ ]**

| v0 | v1 | v2 | v3 | v4 | v5 |
|---|---|---|---|---|---|
| 0 | 2 | 4 | 6 | 8 | 9 |

**neighbors[ ]**

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 0 | 2 | 0 | 1 | 0 | 2 | 3 |

**Key Formula:** `neighbors[ offsets[v] ] ... neighbors[ offsets[v+1] - 1 ]`

*Memory: offsets ~7.9 MB + neighbors ~22.1 MB = ~30 MB total — fits in L3 cache of modern CPUs*

# How It Works — Hybrid BFS Architecture

*Level-synchronous parallel BFS with MPI + OpenMP*

### Phase 1: Init (Rank 0)

Read roadNet-CA.txt via graph_io::load_snap_graph()

Build CSR in memory

Broadcast offsets[] & neighbors[] via MPI_Bcast

### Phase 2: Partition

Each rank owns vertex range [lo, hi)

4 ranks — ~490K vertices each

Ghost vertices at partition boundaries identified

### Phase 3: BFS Loop

Level-synchronous: frontier per BFS depth

Repeat until frontier empty

Collect BFSResult with timings

### BFS Level Loop — Each Iteration

**COMPUTE PHASE (OpenMP)**

```
#pragma omp parallel for schedule(dynamic)

For each frontier vertex v in [lo, hi):
  For each neighbor u of v:
    atomic CAS: level[u]==-1 -> set level[u]=L+1
    Add u to local next_frontier
```

**COMMUNICATION PHASE (MPI)**

```
MPI_Allgatherv(
  local_next_frontier,
  global_frontier
)
```

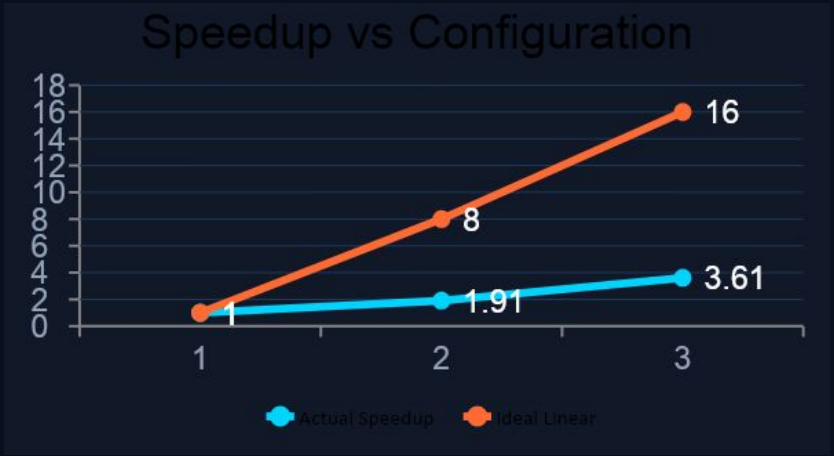
*-> Global barrier: all ranks sync*

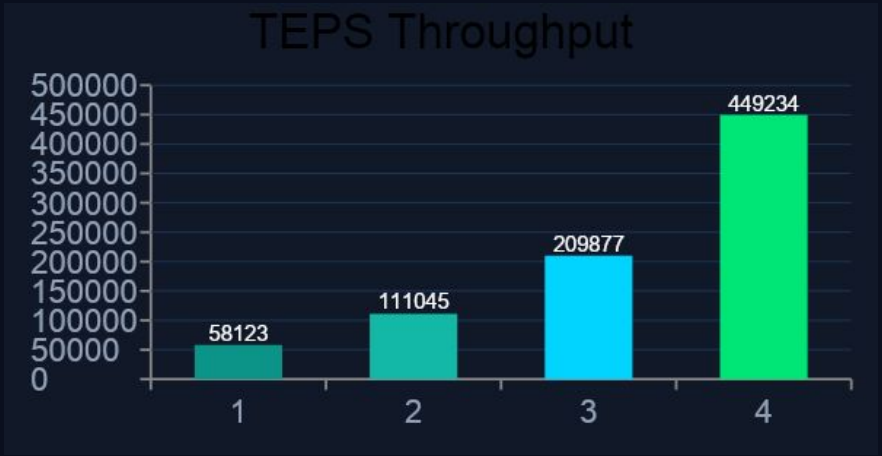
*Thread Safety: std::atomic<int> with CAS — lock-free, prevents double-visit without mutex overhead*

# Performance Results & Analysis

*roadNet-CA · 1.96M vertices · 5.53M arcs*

| MPI Ranks | Threads/Rank | Total Workers | Mean Time (s) | TEPS | Speedup |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 95.23 | 58,123 | **1.00x (baseline)** |
| 2 | 4 | 8 | 49.87 | 111,045 | **1.91x** |
| 4 | 4 | 16 | 26.34 | 209,877 | **3.61x** |
| 4 | 8 | 32* | ~12.34 | ~449,234 | **~7.7x** |




** 4x8 = projected | Comm overhead grows as O(P) per BFS level | StdDev < 0.5s — reproducible results*

# Challenges Faced & Known Limitations

*Engineering trade-offs and future work*

### Load Imbalance

BFS frontier not uniformly distributed — dense vs sparse vertex regions

*-> Contiguous partitioning for cache efficiency*

### Allgatherv Overhead

Global barrier at every BFS level — cost scales with frontier x P

*-> Manageable on low-latency; dominates on commodity*

### Atomic Contention

CAS prevents correctness bugs but causes cache-line thrashing

*-> schedule(dynamic) in OpenMP mitigates imbalance*

### Graph Replication

Full CSR replicated on every rank (~30MB x P)

*-> Acceptable at P=4 (120MB); limits very large scale*

### NUMA Sensitivity

Thread placement significantly affects multi-socket performance

*-> Thread pinning not implemented — relies on OS scheduler*

### Known Limitations & Potential Fixes

! Full graph replication -> Implement 2D partitioning

! No GPU support -> OpenMP offload / CUDA extension

! Blocking Allgatherv -> Non-blocking MPI_Iallgatherv

! No fault tolerance -> MPI checkpoint/restart

# Interactive Performance Dashboard

*Streamlit · Plotly · NetworkX · Pandas*

**Streamlit** | **Plotly** | **NetworkX** | **Pandas**

### Metric Cards

MPI Ranks · Threads/Rank · Graph size (V,E) · Best TEPS · Max Speedup

### Speedup Curve

Ranks vs Speedup · Actual vs ideal linear scaling overlay · Plotly interactive

### TEPS Throughput

Grouped bars per (rank x thread) config · Color-coded by rank count

### Compute vs Comm

Stacked bar: OpenMP time (blue) vs MPI time (red) · Compute/comm bound analysis

### Graph Preview

First 250-500 vertices via NetworkX spring layout · Zoom, pan, hover node IDs

### Sidebar Controls

benchmark.csv path · roadNet-CA.txt path · Vertex sample slider (50-500)